

Order form parsing evaluation task

Order Form Parsing Challenge

Introduction

In SaaS and enterprise software businesses, revenue recognition is a critical accounting process governed by standards like ASC 606. A key input to this process is the **order form** -- a contractual document between a vendor (the selling company) and a customer (the buying company) that outlines what products or services are being sold, at what price, and over what time period.

These order forms come in many shapes and sizes: PDFs, scanned documents, or digital contracts. They vary widely in formatting, terminology, and structure across different companies and even across different deals within the same company. Despite this variation, the core information they contain is consistent: a set of **contract items** (the products or services being sold) along with their associated pricing and scheduling details.

The challenge is to **extract structured data from these unstructured order form documents** using a Large Language Model (LLM).

Problem Statement

Given an order form document (typically a PDF), the goal is to parse it and produce two categories of structured output:

1. **Order Form Details** -- High-level metadata about the contract itself (e.g., effective dates, signing dates, total value).
2. **Contract Items** -- A list of individual line items representing the products or services included in the order form, along with their attributes.

An order form can contain **one or many contract items**. For example, a single order form might include a platform license, a support package, and a professional services engagement -- each as a separate contract item.

What is a Contract Item?

A contract item represents a distinct product or service (sometimes called a SKU) that the selling company offers. Think of it as a line item on an invoice or a row in a pricing table within the order form.

Examples of contract items might include:

- A yearly platform subscription license
- A block of prepaid API calls
- A professional services engagement for implementation
- A premium support package

The Constraint: A Known List of Contract Items

This is not an open-ended extraction task. The selling company maintains a **predefined catalog of products and services** (referred to as the SKU list). Every contract item extracted from an order form **must map to an entry in this catalog**.

You cannot simply ask the LLM "what are the contract items?" and accept whatever it returns. The model must identify which items from the known catalog are present in the order form. If something in the document does not correspond to a known SKU, it should not be extracted as a contract item.

The SKU catalog contains entries with the following information:

- **Name** -- The product or service name (e.g., "Enterprise Platform License")
- **Code** -- A unique identifier for the SKU within the organization
- **Description** -- What the product or service entails
- **Parsing Instructions** -- Specific guidance on how this SKU typically appears in order forms, including common aliases or variations in naming

Attributes of a Contract Item

When a contract item is identified in an order form, the following attributes need to be extracted for each one:

Attribute	Description
SKU Reference	Which product/service from the known catalog this item maps to
Currency	The currency of the pricing (e.g., USD, EUR, GBP)

Attribute	Description
Total Listed Value	The total monetary value of this line item as stated in the contract
Quantity	The number of units being purchased (defaults to 1 if not explicitly stated)
Unit Price	The price per unit, if applicable (may be null if only a total price is given)
Discount	Any discount applied, either as a percentage or an absolute amount
Service Start Date	When the service or product delivery begins
Service End Date	When the service or product delivery ends
Invoicing Schedule Type	How the customer is billed -- one of: upfront, recurring, <u>milestone</u> -based, usage-based, or hybrid
Invoicing Frequency	How often invoices are sent -- one of: monthly, quarterly, yearly, biannually, or one-time
Payment Terms	The payment terms stated in the contract (e.g., "Net 30", "Due on Receipt")
Special Notes	Any additional context or conditions specific to this line item

Example: A Sample Order Form

Refer to the attached file [example_order_form_redacted.pdf](#) for a sample order form. This is a real (redacted) order form from which contract items need to be extracted. Below is a walkthrough of what should be extracted from it.

SKU Catalog for this Example

For this order form, assume the following SKU catalog is provided:

SKU Code	Name	Description
SKU-001	Talent Intelligence Platform Premium Edition	Foundational platform layer integrating with HR/IT systems, providing AI-driven talent insights and workflow automation.
SKU-002	Talent Acquisition	Product suite for finding and hiring talent, covering the full candidate journey from sourcing through selection.

SKU Code	Name	Description
SKU-003	Premium Support	Dedicated support services including priority response times and extended support hours.
SKU-004	Professional Services	Implementation, training, and consulting services.
SKU-005	Data Analytics Add-on	Advanced reporting and data warehouse capabilities.

Expected Extraction

From this sample order form, the system should identify **3 contract items** (not 4 or 5 -- SKU-004 and SKU-005 are in the catalog but not present in this order form):

```
{
  "contract_items_count": 3,
  "contract_items": [
    {
      "name": "Talent Intelligence Platform Premium Edition",
      "sku_reference": "SKU-001",
      "currency": "USD",
      "total_listed_value": 765000.00,
      "quantity": 1,
      "unit_price": 255000.00,
      "unit_price_period": "year",
      "service_start_date": "2023-11-01",
      "service_end_date": "2026-10-31",
      "invoicing_schedule_type": "recurring",
      "invoicing_frequency": "yearly",
      "payment_terms": "Net 55",
      "special_notes": "Up to 25,000 employee size"
    },
    {
      "name": "Talent Acquisition",
      "sku_reference": "SKU-002",
```

```

    "currency": "USD",
    "total_listed_value": 225000.00,
    "quantity": 1,
    "unit_price": 75000.00,
    "unit_price_period": "year",
    "service_start_date": "2023-11-01",
    "service_end_date": "2026-10-31",
    "invoicing_schedule_type": "recurring",
    "invoicing_frequency": "yearly",
    "payment_terms": "Net 55",
    "special_notes": null
  },
  {
    "name": "Premium Support",
    "sku_reference": "SKU-003",
    "currency": "USD",
    "total_listed_value": 152100.00,
    "quantity": 1,
    "unit_price": 50700.00,
    "unit_price_period": "year",
    "service_start_date": "2023-11-01",
    "service_end_date": "2026-10-31",
    "invoicing_schedule_type": "recurring",
    "invoicing_frequency": "yearly",
    "payment_terms": "Net 55",
    "special_notes": null
  }
]
}

```

Key Observations from this Example

- The order form lists prices **per year** across 3 columns (Year 1, Year 2, Year 3). The total listed value for each item is the sum across all years.
- All three items share the same service period, invoicing frequency, and payment terms -- but this is not always the case. Different contract items can have different terms.

- The SKU catalog contains 5 items, but only 3 are present in this order form. The system must not hallucinate the missing ones.
 - Implementation services are mentioned in the document but are handled by a third-party partner, not sold directly. This should **not** be extracted as a contract item.
 - The invoicing schedule section confirms the yearly billing and provides exact invoice dates and amounts, which should be consistent with the extracted contract items.
-

The Task

Your objective is to an agentic system that (think openclaw) that basically accomishes this:

1. Accepts an order form document and a list of known SKUs as input.
2. Uses an LLM to parse the document and extract:
 - The relevant order form metadata (dates, signatories, total value, etc.)
 - All contract items present in the document, with each item mapped to a known SKU and all attributes populated.
 - Reference for claude agent/workflow SDK:
 - <https://www.youtube.com/watch?v=TqC1qOfiVcQ>
3. Returns the extracted data in a structured format.

Considerations

- Accuracy of extracted values (especially monetary amounts and dates) is critical since these feed into downstream accounting processes.
- Order forms are not standardized. The same information may appear in different formats, use different terminology, or be structured differently across documents.
- Multiple contract items may reference the same SKU (e.g., the same product sold for two different time periods).

Other considerations

- The LLM should only extract contract items that match entries in the provided SKU catalog. It should not fabricate items that are not in the catalog.
 - Some attributes may not be explicitly stated in the document (e.g., quantity might be implied as 1, or unit price might need to be derived from total value and quantity).
-

Google drive with link to synthetic data.

Submission:

example_order_form_redacted.pdf

- Create a private repository with your code, and share access with `rhbhatnagar` and `jayant1616` , `codeknight3` , `ajkrish95`
- There should be an readme that clearly mentions results / accuracy + walkthrough of the code.